

STREAMIND: A Real-Time AI Meeting Intelligence Pipeline

Mervegul Parlak
Ozyegin University
Istanbul, Turkiye

Burak Kara
Rennes, France

Alperen F. Zengin,
Ali C. Begen
Ozyegin University
Istanbul, Turkiye

Abstract—Real-time meeting intelligence requires streaming summarization systems that produce coherent outputs under strict latency constraints. We address the Streaming Transcription and Real-Time Extraction for AI Meeting Intelligence Distillation (STREAMIND) Grand Challenge with an eight-node Juturna pipeline. Live Opus/RTP audio enters through the Janus WebRTC gateway, faster-whisper transcribes the stream incrementally, overlapping Automatic Speech Recognition (ASR) chunks are deduplicated and a 300 s rolling window is summarized by *Qwen3.5-4B-AWQ* running under vLLM. Three contributions address the challenge’s joint optimization of summary quality, keyword relevance, and processing latency. Streaming chunk deduplication preserves temporal coherence across overlapping ASR outputs. A keyword guarantee algorithm floors the worst-case keyword penalty at zero. Quality-gated latency optimization through prompt engineering and thinking-mode suppression achieves average processing times of 0.79 s while maintaining summary quality at ceiling.

Index Terms—real-time speech processing, automatic speech recognition, large language models, streaming summarization, WebRTC, Juturna

I. INTRODUCTION

Streaming summarization of live audio—meetings, conference sessions, podcasts—has become a practical target as the volume of recorded speech outpaces the capacity to consume it post hoc. Unlike batch summarization, which operates on complete transcripts after the fact, streaming summarization must produce incremental outputs under strict latency constraints while maintaining coherence across overlapping audio segments. In the STREAMIND Grand Challenge [?], a participating system receives a live Opus-encoded mono audio stream over Real-time Transport Protocol (RTP) and must emit a structured summary with keywords every 300 s, with the scoring weighted across three axes—summary quality, keyword relevance and processing latency.

The challenge defines a per-chunk composite score $C_i = B_i + K_i + L_i$ that creates a non-trivial optimization landscape. B_i sums five Likert-scale quality criteria (factual consistency, relevance, coherence, fluency and conciseness), ranging from 5 to 25. $K_i = 2 \cdot (N_{\text{rel}} - N_{\text{irr}})$ evaluates each of exactly three keywords, ranging from -6 to $+6$. Although the latency bonus $L_i = 10 \cdot e^{-0.5 \cdot t_{\text{proc}}}$ rewards fast processing, it is *gated* on $B_i \geq 10$. A pipeline that fails to clear the quality floor receives no latency credit at all, regardless of how fast it runs (Fig. 2). This creates a hard cliff—a pipeline optimized for raw speed that produces $B_i = 9$ scores strictly worse than a slower

pipeline achieving $B_i = 15$. A flat bonus of $+4$ is awarded for using the Janus Web Real-Time Communications (WebRTC) gateway [1] as the audio source. In practice the pipeline drives B_i comfortably above the quality gate and then optimizes L_i at fixed B_i , rather than chasing additional summary quality through larger models. Beyond the gate, the marginal B_i a larger model can deliver fails to compensate for the L_i it gives up (Fig. 4). A regression in K_i is treated as independently unacceptable, since the asymmetric per-keyword penalty erases multiple points of latency improvement.

We make three contributions, each targeting a specific scoring term:

- 1) A streaming chunk deduplication strategy that preserves temporal coherence across overlapping 5 s Automatic Speech Recognition (ASR) chunks while keeping the aggregated 300 s window non-redundant.
- 2) A keyword guarantee algorithm that strips banned generic terms and back-fills missing slots from transcript proper nouns, converting the worst-case $K_i = -6$ into a floor of $K_i = 0$.
- 3) Quality-gated latency optimization through prompt engineering and thinking-mode suppression, achieving average processing times of 0.79 s ($L_i \approx 6.8$) while maintaining B_i at 24–25 of 25.

Section II surveys related work and background. Section III details the pipeline architecture. Section IV covers deployment. Section V presents evaluation results. Section VI discusses limitations and future directions.

II. BACKGROUND AND RELATED WORK

Juturna [2] is an open-source Python library for real-time multimedia Artificial Intelligence (AI) pipelines. Pipelines are declared in JSON as directed node graphs; Juturna compiles each declaration into a running dataflow and manages every node through a four-stage lifecycle (`configure()` $\rightarrow$ `warmup()` $\rightarrow$ `start()` $\rightarrow$ `stop()`). Nodes exchange typed messages along directed edges; roles are *source* (data ingestion), *processing* (transform), and *sink* (delivery). Janus [1] is a general-purpose WebRTC server; Juturna’s built-in `audio_rtp` source subscribes to its RTP endpoint, receiving mono Opus/48 kHz audio [3], [4] with identical code paths in development and evaluation, earning the flat $+4$ bonus.

For ASR, the Whisper model family [5] has demonstrated strong transcription performance across languages and acoustic conditions. We use faster-whisper [6], which reimplements Whisper inference via CTranslate2 with int8 quantization for reduced memory footprint and latency. In streaming mode, audio is chunked into short overlapping segments and transcribed independently with previous-text conditioning disabled to prevent context drift across chunks.

Small instruction-tuned Large Language Models (LLMs) can produce structured JSON outputs (summaries and keywords) in a single inference call [7]. vLLM [8] provides efficient Graphic Processing Unit (GPU) inference through paged attention and continuous batching, enabling in-process model loading without daemon overhead. Quality is evaluated by LLM-as-judge methods [9], which score generated summaries against the source transcript on multiple criteria. Single-model bias is mitigated by drawing the judge panel from several model families. We position STREAMIND as a *systems* contribution that composes these building blocks into a scoring-formula-aware pipeline, rather than proposing new models or training procedures.

III. SYSTEM ARCHITECTURE

Figure 1 shows the eight-node pipeline: each node maps to one or more terms of the scoring formula $C_i = B_i + K_i + L_i$. The audio source secures the +4 Janus bonus; hallucination filtering and novel extraction raise the transcript quality seen by the summarizer, improving B_i ; the summarizer trades B_i against L_i ; keyword validation alone acts on K_i .

A. Audio Reception

Janus [1] accepts the challenge WebRTC session and forwards mono Opus/48kHz RTP media to Juturna’s built-in `audio_rtp` source node on User Datagram Protocol (UDP) port 8888. The pipeline declares `payload_type: 97` and `encoding_clock_chan: "opus/48000/1"` matching the challenge specification. Using Janus in both development and evaluation ensures identical code paths and earns the flat +4 bonus at zero architectural cost.

B. Incremental ASR

The `audio_chunker` node buffers the continuous audio stream into 5s chunks with 1s overlap, yielding a 4s advance per step (Fig. 3). Each chunk is transcribed by the `transcriber_whisper` node running faster-whisper [6] at float16 precision on the GPU, co-resident with the summarizer. The distilled `large-v3-turbo` model adds approximately 2 GB of Video Random Access Memory (VRAM) alongside the int4-Activation-aware Weight Quantization (AWQ) summarizer, fitting comfortably within the 24 GB development card with headroom for the KV cache. Previous-text conditioning is disabled to prevent hallucination propagation across chunks, and Voice Activity Detection (VAD) filtering via Silero [10] skips silence segments.

An exploratory comparison of two model variants on 19 30s windows of a 36-minute podcast episode (Table II) measured `small.en` (244M parameters, English-only) against

`large-v3-turbo` (809M, multilingual distillation of `large-v3`). Both achieved identical average Word Error Rate (WER) (0.69), with negligible composite-score differences at 30s granularity. We adopted `large-v3-turbo` for production because of its broader vocabulary coverage on long-form podcast content, with per-window quality at 300s granularity reported in Section V-C.

C. Hallucination Filter

The `hallucination_filter` node applies two post-ASR guards. First, inline noise tags (`[Music]`, `(applause)`) are stripped via regex patterns. Second, the cleaned text is checked against 22 known hallucination patterns (e.g., “thank you for watching”, “please subscribe”, common filter tokens) and silently dropped if matched. Without this filter, hallucinated sentences reach the summarizer and are faithfully summarized as facts, directly harming B_i factual consistency.

D. Novel Chunk Extraction

The 1s overlap between consecutive chunks means that roughly one second of speech appears in both the previous and current transcription. The `novel_extractor` node performs word-level overlap detection: for overlap lengths from 10 words down to 1, it compares the suffix of the previous chunk with the prefix of the current chunk. An exact match returns the overlap length immediately; a `SequenceMatcher` fallback with similarity threshold 0.8 handles minor ASR variations in the overlap region (e.g., “the quick brown” vs. “the quicker brown”). Only the novel words beyond the detected overlap are emitted. Without this deduplication, the 300s window would contain approximately 20% redundant text, wasting context budget and biasing summaries toward repeated phrases.

E. Window Aggregation

The `window_aggregator` node concatenates novel transcript segments into a rolling 300s context window. A window is flushed when $t_{\text{chunk_end}} \geq t_{\text{window_start}} + 300$. On pipeline shutdown, partial windows are emitted to avoid data loss at stream end.

F. Summarization

At warmup, the `summarizer_vllm` node loads model weights from a local filesystem path through vLLM’s in-process LLM class [8]. Loading the model in-process removes the need for an inference daemon, intermediate middleware, or any network dependency at runtime. The inference engine selection rationale is detailed in Section IV-A.

We use Qwen3.5-4B [7], [11] quantized to int4 via AWQ [12] at 4096 context length. AWQ quantization roughly quarters the weight footprint relative to bfloat16, reducing inference latency and preserving KV-cache headroom for 300s windows within the 24 GB development GPU.

The summarization prompt is designed with explicit alignment to the five Likert evaluation criteria. “2–3 sentences

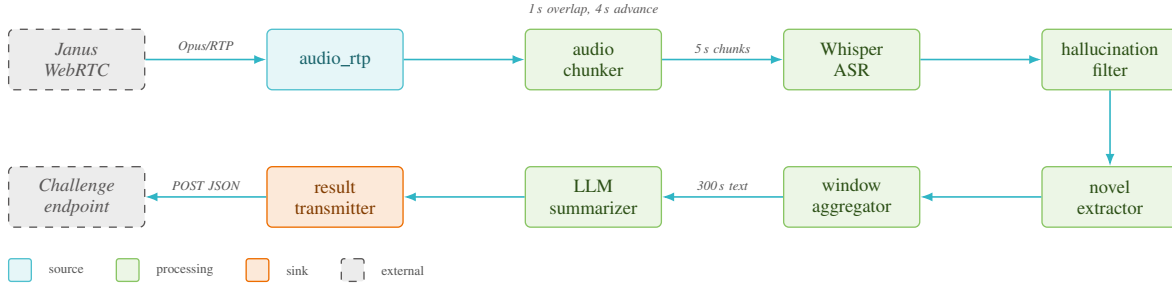


Fig. 1: End-to-end STREAMIND pipeline. Janus forwards Opus/RTP media into Juturna’s built-in `audio_rtp` source node; chunks slide every 4 s through ASR; deduplication and hallucination filtering precede a 300 s aggregation buffer; the summarizer emits `{summary, keywords[3]}` per window; the sink stores locally and POSTs to the challenge endpoint.

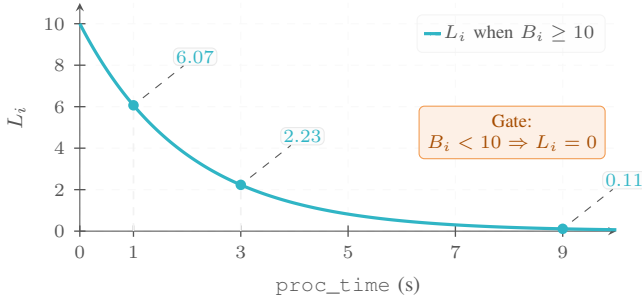


Fig. 2: Latency bonus $L_i = 10e^{-0.5 \cdot t_{\text{proc}}}$, gated by $B_i \geq 10$. Below the gate, latency contributes zero; above it, returns diminish rapidly past 2 s.

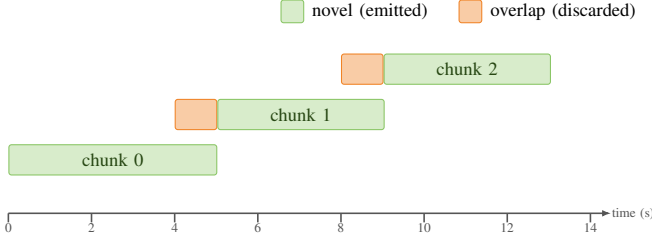


Fig. 3: Audio chunking and novel extraction. Five-second chunks slide by 4 s; the novel extractor isolates newly-spoken content each step, discarding the 1 s overlap with the previous chunk.

capturing the main topics and conclusions” targets *conciseness* and *coherence*. “Only state facts explicitly present in the transcript. Do not infer” targets *factual consistency*. “Exactly 3 keywords [...] Never use generic terms like ‘discussion’, ‘meeting’, or ‘topic’” targets keyword relevance (K_i). Chain-of-thought generation is suppressed (`enable_thinking=False`), directly reducing output token count and thus t_{proc} . Sampling parameters are set to temperature 0.15, top- $p = 0.9$, repetition penalty 1.05, and `max_tokens = 384`.

Two failure modes are handled without emitting an empty summary. When the transcript is shorter than 80 characters (end-of-stream fragments), the LLM is skipped. When the call raises an exception, returns unparseable JSON, or produces

an empty summary, the pipeline falls back to an *extractive summary*: the first sentence(s) of the transcript, capped at 300 characters. This verbatim excerpt guarantees high factual consistency (B_i cannot be penalized for a direct quote) and costs approximately 0 ms, preserving L_i . A warmup generation call during startup triggers CUDA kernel compilation before the first real window, preventing a cold-start latency spike.

G. Keyword Validation

Regardless of LLM output quality, the `ensure_three_keywords` function (Algorithm 1) guarantees exactly three keywords per window. The algorithm first removes empty entries and filters against a set of banned generic terms (“meeting”, “discussion”, “topic”, *etc.*). Case-insensitive deduplication preserves first-seen casing. If fewer than three keywords remain, candidates are extracted from the transcript ranked by token frequency, with ties broken toward capitalized tokens (likely proper nouns). As a last resort, conservative fallback terms are appended.

The range of K_i is effectively tightened from $[-6, +6]$ to $[0, +6]$. A summarizer that produces no usable keywords on a given window now scores $K_i = 0$ instead of $K_i = -6$, removing the largest downside risk in the scoring formula.

Algorithm 1 Keyword guarantee: `ensure_three_keywords`.

Require: kw : keyword list from LLM, T : transcript text

Ensure: exactly 3 keyword strings

- 1: $kw \leftarrow$ strip whitespace, remove empty entries
- 2: $kw \leftarrow \{w \in kw \mid w \notin \text{BANNED}\}$
- 3: $kw \leftarrow \text{deduplicate}(kw, \text{case-insensitive})$
- 4: **if** $|kw| < 3$ **then**
- 5: $cand \leftarrow$ rank tokens in T by freq, capitalization
- 6: filter $cand$: remove stopwords $\cup$ BANNED
- 7: append $cand$ to kw until $|kw| = 3$
- 8: **end if**
- 9: **if** $|kw| < 3$ **then**
- 10: pad kw with fallback terms to length 3
- 11: **end if**
- 12: **return** $kw[:3]$

H. Result Transmission

The `result_transmitter` sink node maps internal keys to the challenge contract (`window_start` $\rightarrow$ `from`, `window_end` $\rightarrow$ `to`, `latency` $\rightarrow$ `proc_time`) and writes each window as `window_N.json`. When `destination_endpoint` is configured, results are also POSTed via HTTP. A debug mode persists the full transcript and raw LLM output alongside each window for post-hoc analysis.

IV. IMPLEMENTATION AND DEPLOYMENT

Two JSON files together define a pipeline configuration. All nodes except the summarizer are declared in `config-base.json` and a profile drawn from `pipelines/summarizer/` supplies the model-specific parameters such as the model path, sampling configuration and VRAM budget. A merge tool combines these at launch time and a single shell script serves as the entry point for both development and submission.

The submission Docker image is based on NVIDIA CUDA 12.4 runtime with Python 3.12 and `uv` for dependency management. Model weights for both the ASR model and the summarizer are baked into the image at build time, ensuring zero network dependency at runtime. The `docker-compose.yml` file orchestrates Janus and the pipeline container as two services, with Janus forwarding RTP audio to the pipeline container on UDP port 8888. Table I summarizes all pipeline nodes and their key parameters.

A. Inference Engine

We select vLLM [8] over alternative serving frameworks for three reasons aligned with the scoring formula and submission constraints. First, the in-process library Application Programming Interface (API) eliminates the need for a separate inference server daemon, so the submission container runs as a single process with no independent health-check or fail mode. Second, paged attention holds t_{proc} low across varying transcript lengths, which is the direct lever on $L_i = 10 e^{-0.5 \cdot t_{\text{proc}}}$. Third, the explicit `gpu_memory_utilization` parameter caps VRAM usage, enabling the int4-AWQ summarizer to co-reside with the ASR model within the 24 GB development card.

Two alternatives were considered. SGLang [13] (RadixAttention prefix caching) offers roughly 29% higher throughput than vLLM as a near drop-in replacement and remains a future upgrade path if L_i becomes the binding constraint (Section VI). TensorRT-LLM achieves the highest raw throughput but requires per-model ahead-of-time compilation, which conflicts with iterative model selection during development and imposes significant overhead on the single-GPU submission environment. vLLM’s broader model coverage and zero compilation overhead best match the challenge’s single-GPU, rapid-iteration context.

TABLE I: Pipeline nodes with key parameters and scoring targets.

Node	Type	Key parameters	Target
audio_rtp	source	Opus/48k, port 8888	+4
audio_chunker	proc	5 s chunk, 1 s overlap	—
transcriber	proc	float16, GPU, VAD on	B_i
halluc. filter	proc	22 exact patterns	B_i
novel_extractor	proc	10 words, sim ≥ 0.8	B_i
window_agg.	proc	300 s rolling	—
summarizer	proc	Qwen3.5-4B-AWQ [11], $T=0.15$	B_i, L_i
transmitter	sink	JSON + HTTP POST	—

V. EVALUATION

A. Setup

The current results cover three podcast episodes from the Rev16 dataset [14]; full evaluation on the challenge audio set is ongoing and will expand this coverage. A 300 s window pipeline run produces roughly five scored windows per episode. To absorb LLM-judge non-determinism, every model comparison repeats the pipeline at least twice per configuration. The numbers reported throughout are means across these runs, accompanied by standard deviations where more than one run is available. The offline judge harness uses a cross-family panel of three int4-AWQ judges (Mistral-Small-24B-Instruct, Phi-4, and Gemma-3-27B-it) chosen from outside the Qwen family to avoid self-evaluation bias. Each judge runs in a separate subprocess that exits before the next one starts, allowing VRAM to be reclaimed between runs since the 24 GB development GPU cannot host the full panel at once. Following the G-Eval paradigm [9], each judge scores every summary against the five Likert criteria defined by the challenge and evaluates keyword relevance as a per-keyword boolean flag. Panel consensus is reported as the mean and standard deviation across the three judges. Low dispersion lends confidence to a score, while high dispersion flags judge-specific bias and is treated with caution. All experiments ran on an NVIDIA RTX 4090 with 24 GB of VRAM under CUDA 12.4, while the submission target is the larger RTX Pro 4500 with 32 GB.

WER is measured independently using jiwer against ground-truth transcripts. Because the ground-truth text lacks timestamps, we employ proportional character-based alignment with a sliding-window search ($\pm 30\%$ offsets) to find the best-matching reference segment for each window. WER is an independent signal of ASR quality and is not part of the $B_i/K_i/L_i$ scoring.

B. ASR Model Comparison

Table II reports an exploratory comparison of two faster-whisper variants across 19 30 s windows from a single rev16 episode, collected during early pipeline development before the production switch to `large-v3-turbo` at 300 s window granularity (Section V-F). Both models achieve identical average WER (0.69), yet `small.en` produces a slightly higher final score (40.00 vs. 39.73). The 0.27-point gap traces back

to keyword stability rather than transcription accuracy. The smaller model reached $\bar{K} = 4.9$ against 4.5 for the larger one, with the difference concentrated in windows where the larger model’s transcriptions led the summarizer to keywords that the judge rated as less relevant. This suggested early on that transcription accuracy is not the pipeline bottleneck and motivated the 300s-window evaluation in Section V-C, where the larger model’s broader vocabulary coverage on long-form podcast content justified the production choice.

TABLE II: Exploratory ASR comparison from early pipeline development (19 30s windows, single rev16 episode, Qwen3.5-4B-AWQ summarizer). Production pipeline uses large-v3-turbo at 300s granularity; see Table III. Higher is better for B , K , L , C ; lower for t_{proc} and WER.

Metric	small.en	large-v3-turbo
Avg B_i	23.6	23.8
Avg K_i	4.9	4.5
Avg L_i	7.42	7.37
Avg C_i	36.00	35.73
Final score ($\bar{C} + 4$)	40.00	39.73
Avg t_{proc} (s)	0.60	0.62
Avg WER	0.69	0.69

C. End-to-End Scoring

Table III reports per-window scores for Qwen3.5-4B-AWQ on a 25-minute podcast episode (5 windows of 300s each), averaged across two independent runs to reduce LLM-judge variance. All five windows achieve $K_i = 6.0$ (the maximum), confirming that the keyword guarantee reliably fills all three slots with relevant terms on every window. B_i remains narrowly bracketed between 24.0 and 24.7 out of 25, close to the Likert ceiling, and run-to-run variation in C_i never exceeds 0.9 points on any single window, which we attribute to judge non-determinism rather than to instability in the pipeline. The latency component shows the widest spread in the table and ranges from $L_i = 6.05$ on window 3 to $L_i = 7.18$ on window 4. Window 3 spans a dense 300s segment that took 1.01s on average to summarize, while window 4 covers only the trailing 105s of audio and was therefore compressed in 0.66s. Averaging C_i across the five windows yields $\bar{C} = 37.1$, which combined with the Janus bonus produces a final source score of **41.09** for this episode. Cross-domain validation on two Internet Engineering Task Force (IETF) 125 sessions (Computing-Aware Traffic Steering; Media over Quick UDP Internet Connections (QUIC)) produced scores of 40.19 and 40.24, closely tracking the Rev16 mean (Section V-F) despite denser technical vocabulary.

D. Outlier Analysis

At 300s granularity, topic transitions and informal speech are averaged across a full five-minute window. Within Table III, B_i stays narrowly between 24.0 and 25.0 and K_i is uniformly 6.0, so C_i differences between windows are driven entirely by latency. The primary outlier risk is a dense-speech

TABLE III: Per-window scores, Qwen3.5-4B-AWQ [11], 300s windows (ep. 27, 2-run mean, cross-family panel consensus). t = avg processing time (s). Preliminary.

Win.	From	To	t (s)	B_i	K_i	L_i	C_i
0	0	300	0.86	24.7	6.0	6.51	37.2
1	300	600	0.72	24.7	6.0	6.99	37.7
2	600	900	0.85	24.0	6.0	6.53	36.5
3	900	1200	1.01	24.3	6.0	6.05	36.4
4	1200	1305	0.66	24.5	6.0	7.18	37.7
avg			0.82	24.4	6.0	6.65	37.1

window that inflates t_{proc} (window 3: 1.01s, $L_i = 6.05$), not a content-quality failure. At 30s granularity, topic transitions produce B_i drops of 3–6 points and K_i swings as large as -4 , confirming that 300s aggregation materially reduces per-window variance.

E. Ablations

Each upstream node defends a specific scoring term. Without the `hallucination_filter`, 22 matched ASR artifact phrases (e.g., “thank you for watching”) reach the summarizer and reduce the judge’s factual consistency score. Without the `novel_extractor`, the window contains $\approx 20\%$ repeated text from chunk overlap, biasing summaries toward repeated phrases. Without Algorithm 1, the K_i range reverts to $[-6, +6]$; the cross-family sweep (Section V-F) shows this is non-hypothetical: Llama-3.1-8B requires the guarantee to hold $\bar{K}_i = 6$ on every window.

F. Model-Size and Cross-Family Sweep

Figure 4 decomposes scores for six configurations across three Rev16 episodes (6–15 runs, 300s windows). Within the Qwen3.5 family, $B_i + K_i$ is nearly flat across all four variants; score differences are driven entirely by latency, with t_{proc} ranging from 0.79s (4B-AWQ) to 2.51s (9B-BF16). AWQ reduces t_{proc} by 35% at 4B and 30% at 9B, yielding source scores of 40.14 ± 0.85 , 38.97 ± 0.91 , 37.15 ± 1.53 , and 36.12 ± 1.42 respectively. Gemma-3-12B-IT-AWQ [15] scores 35.63 ± 1.06 despite $3\times$ more parameters ($\bar{B}_i \approx 22.1$ vs. the Qwen mean of 29.1), confirming that instruction-tuning alignment matters more than scale. Llama-3.1-8B-Instruct [16] scores 32.31 ± 4.59 ; episode-to-episode $\bar{B}_i$ swings from ≈ 13 to ≈ 24 trace to content-driven instability the keyword guarantee cannot resolve. Qwen3.5-4B-AWQ is selected as the production summarizer.

G. Threats to Validity

Several limitations affect the reported numbers. Our offline judge panel of Mistral-Small-24B, Phi-4, and Gemma-3-27B (all int4-AWQ) is only a local approximation of the hidden challenge judge. Even where the three judges agree, the reported scores should be read as directional rather than absolute. LLM judges are also non-deterministic. Repeated runs on identical inputs produce C_i values that differ by 0.1–0.9 points, and content variation across audio contributes a

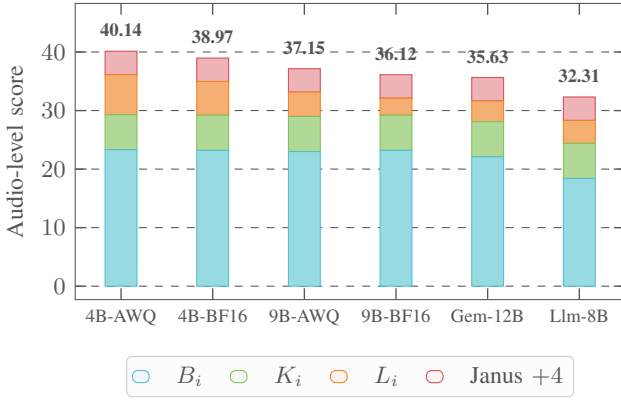


Fig. 4: Score decomposition across six configurations (cross-audio mean, three Rev16 episodes, 300 s windows, RTX 4090). *Gem-12B* = Gemma-3-12B-IT-AWQ; *Llm-8B* = Llama-3.1-8B-Instruct. $B_i + K_i$ is flat within the Qwen family; score differences come entirely from L_i .

further 0.9–1.5 points of standard deviation. Averaging across runs reduces this noise but does not eliminate it. Evaluation so far covers three podcast episodes from the Rev16 dataset and two IETF 125 technical sessions (Section V-C); the held-out challenge audio may differ in speaker count, topic complexity, and acoustic conditions, so rankings may shift as coverage grows. WER measurement relies on proportional character alignment against untimestamped ground truth, which introduces alignment noise on top of the underlying transcription errors. The development hardware (RTX 4090, 24 GB) is more VRAM-constrained than the submission target (RTX Pro 4500, 32 GB), so models sized to fit the development card fit the submission card with headroom.

VI. DISCUSSION

The latency gate on L_i matches a real user preference, since a slow but accurate summary is more useful than a fast but wrong one and the scoring formula refuses to reward the latter case. Keyword scoring has the opposite shape. Each irrelevant keyword carries a -2 point cost out of a six-point range, so one bad keyword in three has roughly the same impact as the run-to-run judge variance reported in Section V-G. Algorithm 1 responds to that asymmetry by falling back to conservative, transcript-derived keywords whenever the LLM fails to provide three usable terms.

The current summarization prompt is static across all windows. Two directions extend this work. First, latency reduction at fixed quality: migrating to SGLang [13] (reported at 29% higher throughput than vLLM as a near drop-in replacement) would lift L_i without affecting B_i , and mixture-of-experts models could approach large-model quality at small-model decode speed. Second, robustness: structured-output decoding removes the JSON parsing failure mode entirely, and content-adaptive prompting that detects topic transitions could reduce the B_i variance observed on dense windows and in cross-family comparisons.

VII. CONCLUSION

We presented an eight-node Juturna pipeline for the STREAMIND Grand Challenge that processes live Opus/RTP audio into structured summaries and keywords in real time. Three design choices drove the reported numbers. Deduplicating overlapping ASR chunks before aggregation preserved transcript coherence and protected B_i . The keyword guarantee algorithm held K_i at its maximum of 6 on every evaluated window and eliminated the worst-case keyword penalty entirely. Prompt engineering, thinking-mode suppression, and int4-AWQ quantization brought average processing time down to 0.79 s ($L_i \approx 6.8$), while the cross-family judge panel scored $B_i = 24$ –25 on every window. On current preliminary data (3 Rev16 episodes), the final source score for Qwen3.5-4B-AWQ at 300 s windows averages 40.14 ± 0.85 across episodes and runs, including the Janus bonus.

REFERENCES

- [1] Meetecho, “Janus: An open source, general purpose, WebRTC server,” [Online] Available: <https://github.com/meetecho/janus-gateway>, 2014, Accessed on June 1, 2026.
- [2] —, “Juturna: A python library for fast data pipeline prototyping,” [Online] Available: <https://github.com/meetecho/juturna>, 2024, Accessed on June 1, 2026.
- [3] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A transport protocol for real-time applications,” IETF, Tech. Rep. RFC 3550, 2003.
- [4] J.-M. Valin, K. Vos, and T. B. Terriberry, “Definition of the Opus audio codec,” IETF, Tech. Rep. RFC 6716, 2012.
- [5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *International conference on machine learning*. PMLR, 2023.
- [6] G. Klein, “faster-whisper: Faster whisper transcription with CTranslate2,” [Online] Available: <https://github.com/SYSTRAN/faster-whisper>, 2023, Accessed on June 1, 2026.
- [7] A. Yang, A. Li *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [8] W. Kwon, Z. Li *et al.*, “Efficient memory management for large language model serving with pagedattention,” in *Proc. SOSP*, 2023.
- [9] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. EMNLP*, 2023.
- [10] Silero Team, “Silero VAD: pre-trained enterprise-grade voice activity detector (VAD), number detector and language classifier,” [Online] Available: <https://github.com/snakers4/silero-vad>, 2024, Accessed on June 1, 2026.
- [11] cyankiwi, “Qwen3.5-4B-AWQ-BF16-INT4: int4-AWQ quantisation of Qwen3.5-4B,” [Online] Available: <https://huggingface.co/cyankiwi/Qwen3.5-4B-AWQ-BF16-INT4>, 2025, Accessed on June 1, 2026.
- [12] J. Lin, J. Tang *et al.*, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [13] L. Zheng, L. Yin *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [14] Rev, “Rev16: Long-form podcast transcription evaluation set,” [Online] Available: <https://huggingface.co/datasets/distil-whisper/rev16>, 2024, Accessed on June 1, 2026.
- [15] G. Ernst, “Gemma-3-12B-IT-AWQ: int4-AWQ quantisation of Gemma-3-12B-IT,” [Online] Available: <https://huggingface.co/gaunernst/gemma-3-12b-it-awq>, 2025, Accessed on June 8, 2026.
- [16] Meta AI, “Llama-3.1-8B-Instruct,” [Online] Available: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, 2024, Accessed on June 8, 2026.